

Kafka Incident Monitoring: 장애 대응을 운영 지식으로 바꾸기

로그를 모으는 시스템이 아니라, 장애 원인 후보와 확인 순서를 제공하는 운영 지식 플랫폼

요약

ERP·Groupware·ESS에서 발생하는 운영 이벤트와 장애 이벤트를 Kafka로 중앙 수집하고, 고객사·업무 도메인·배포 버전·설정 변경·재처리 가능 여부를 기준으로 원인 후보를 좁히는 시스템을 구축했다. 이 시스템은 장애 대시보드가 아니라 장애 대응의 표준 진입점으로 설계했다.

핵심 판단은 명확했다. 장애 대응은 서버 로그 검색으로 시작하면 개인의 경험에 의존하게 된다. 운영 시스템은 장애를 구조화된 데이터로 남기고, 다음 대응의 확인 순서를 제공해야 한다.

문제 정의

다중 고객사 ERP 환경에서는 장애가 단일 로그 파일로 설명되지 않는다. 고객사별 설정, DB 상태, 배치 이력, 외부 연동, 권한, 세션, 배포 버전이 함께 영향을 준다. 장애가 발생하면 개발자는 서버 로그, DB 상태, 운영자 문의, 과거 장애 기억을 조합해야 했다.

가장 큰 문제는 해결 지식이 시스템 안에 남지 않는다는 점이었다. 한 개발자가 어렵게 해결한 장애가 다른 고객사에서 다시 발생하면 동일한 추적 비용이 반복된다. 고객사명만 다르고 원인은 같은 장애, 특정 배포 이후 증가한 장애, 특정 설정 조합에서만 발생하는 장애를 사람이 기억으로 연결하는 구조는 지속 가능하지 않았다.

따라서 목표를 “로그 수집”으로 두지 않았다. 목표는 장애 이벤트를 운영 데이터로 바꾸고, 원인 후보·영향 범위·유사 이력·초기 확인 순서를 제공하는 것이었다.

책임 경계

계층	책임
업무 서비스	장애·운영 이벤트를 표준 스키마로 발행한다.
Kafka	업무 transaction과 장애 처리 흐름을 분리한다.
Consumer	저장, 알림, 통계, 유사 장애 매칭, DLQ 처리를 독립적으로 수행한다.

계층	책임
운영 UI	고객사, 업무, 배포, 설정 변경, 재처리 가능 여부 기준으로 장애를 보여준다.
룰/힌트 계층	장애 원인을 단정하지 않고 확인 순서를 제안한다.

핵심 설계 결정

문자열 로그가 아니라 구조화 이벤트를 사용했다

문자열 로그는 검색에는 유리하지만 분류와 자동 힌트 제공에는 한계가 있다. 장애 이벤트에는 다음 기준을 포함했다.

```
tenantId, service, module, domain, severity, errorCode,
requestId, userId, traceId, batchId, deployVersion,
databaseType, retryable, occurredAt, summary, detailHint
```

이 구조를 통해 장애를 고객사, 업무, 모듈, 배포 버전, 재처리 가능 여부로 분류했다. 운영 데이터는 나중에 수집하는 부가 로그가 아니라 실행 흐름의 일부로 설계해야 한다는 기준도 여기서 정리되었다.

Kafka로 수집과 처리를 분리했다

장애 이벤트를 DB에 직접 insert하는 방식은 구현 비용이 낮다. 그러나 장애가 폭주하는 순간 저장소가 병목이 되고, 업무 transaction을 방해할 수 있다. Kafka를 중심 버퍼로 두면 발행과 처리를 분리할 수 있고, 저장·알림·통계·힌트·DLQ consumer를 독립적으로 확장할 수 있다.

이 선택의 기준은 처리량 자체가 아니었다. 장애 수집이 장애를 키우지 않아야 했고, 장애 이벤트를 목적별 consumer가 독립적으로 처리해야 했다.

자동 판정이 아니라 확인 순서를 제공했다

ERP 운영 장애는 복합 원인이 많다. 시스템이 원인을 단정하면 빠르게 보이지만, 잘못된 단정은 대응 시간을 늘린다. 따라서 룰 기반으로 “최근 배포된 mapper 확인”, “고객사 설정 누락 확인”, “재처리 가능 이벤트 확인”, “외부 연동 지연 확인”처럼 확인 순서를 제공했다.

이 방식은 개발자를 대체하는 자동화가 아니라, 개발자가 올바른 지점으로 빠르게 이동하도록 돕는 운영 보조 체계다.

버린 선택과 이유

가장 쉬운 방식은 각 서비스에서 장애가 발생할 때 운영 DB에 바로 기록하는 것이었다. 이 방식은 빠르게 구현할 수 있지만 장애가 물리는 순간 저장소 부하가 업무 transaction으로 역류할 수 있다. 장애 기록 시스템이 다시 장애 원인이 되는 구조는 피해야 했다.

문자열 로그를 더 많이 남기는 방식도 충분하지 않았다. 로그 양이 늘어나도 고객센터, 배포 버전, 설정 변경, 재처리 가능 여부가 분리되지 않으면 운영자는 결국 grep과 기억에 의존한다. 그래서 로그 수집량보다 이벤트 스키마의 해석 가능성을 우선했다.

자동 root cause 판정도 범위를 제한했다. ERP 장애는 설정, 데이터, 배포, 외부 연동이 겹치는 경우가 많다. 시스템이 원인을 단정하면 빠르게 보일 수 있지만, 잘못된 단정은 대응을 왜곡한다. 따라서 자동 결론보다 확인 순서와 유사 이력 제공을 우선했다.

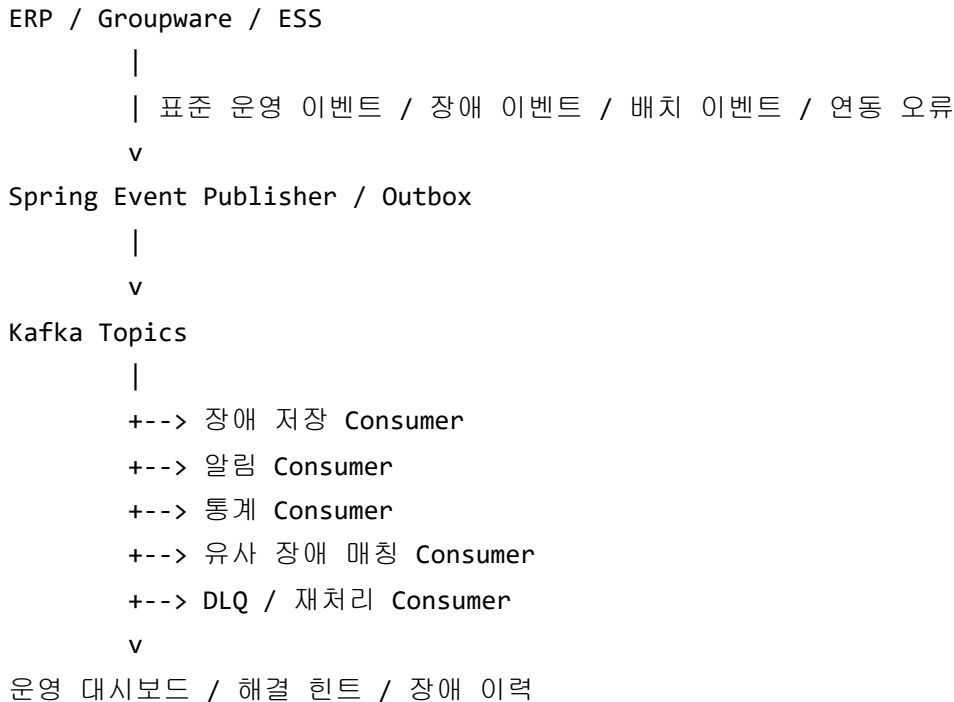
운영자가 확인해야 하는 흔적

이 시스템에서 중요한 화면은 전체 장애 건수 차트가 아니었다. 운영자는 먼저 다음 질문에 답할 수 있어야 했다.

확인 항목	이유
어느 고객센터에서 반복되는가	고객사 설정·데이터·외부 연동 차이를 분리하기 위해서다.
어느 업무 도메인인가	권한, 배치, 연동, 화면 오류의 담당 경계가 다르기 때문이다.
어느 배포 이후 증가했는가	코드 변경과 운영 설정 변경을 함께 봐야 한다.
같은 errorCode가 과거에 있었는가	신규 장애인지 반복 장애인지에 따라 대응 방식이 달라진다.
재처리 가능한가	즉시 복구 가능한 장애와 개발 수정이 필요한 장애를 구분한다.

이 기준을 대시보드와 알림 정책에 반영해야 장애 대응이 개인의 경험에서 시스템의 기록으로 이동한다.

아키텍처



Topic은 이벤트 성격과 consumer의 처리 목적에 따라 분리했다. API 장애, 배치 실패, 외부 연동 오류, DB 정합성 오류, 인증·권한 오류, 성능 지연 이벤트를 단일 topic에 몰아넣지 않았다. 운영자가 확인해야 하는 기준과 처리량, 보관 기간, 알림 정책이 달랐기 때문이다.

운영 검증

가장 먼저 보정한 부분은 알림 정책이었다. 모든 장애를 알리면 초기에는 적극적으로 보이지만 곧 알람 피로가 발생한다. 동일 장애를 묶고, 반복 횟수·영향 고객사 수·심각도·재처리 가능 여부를 기준으로 알림을 제한했다.

두 번째 보정은 이벤트 스키마였다. 로그 양을 늘리는 방향은 장애 분석 비용을 줄이지 못했다. 고객사, 업무 도메인, 오류 코드, 요청 ID, 배포 버전, 재처리 가능 여부처럼 분류 가능한 데이터가 있어야 통계와 힌트가 의미를 갖는다.

세 번째 보정은 대시보드 구성이다. 차트 중심 화면보다 “어느 고객사에서, 어느 업무가, 어느 배포 이후, 얼마나 반복되는가”를 확인하는 화면이 운영자에게 더 유용했다.

운영 결과

적용 이후 장애 대응의 출발점이 개별 서버 접속에서 구조화된 모니터링 화면으로 이동했다. 개발자와 운영자는 먼저 고객사, 업무 도메인, 장애 유형, 최근 배포 여부, 유사 이력을 확인한 뒤 필요한 로그와 DB 상태로 내려갔

다.

반복 장애의 원인과 조치 방법이 운영 지식으로 축적되면서 신규 개발자도 과거 장애를 추적할 수 있는 기반이 마련되었다. 특정 고객센터에서만 발생하는 문제, 특정 배치 이후 반복되는 문제, 특정 외부 연동에서만 발생하는 문제를 더 빠르게 분리했다.

역할과 책임

문제 정의, 장애 이벤트 모델, Kafka 기반 수집 구조, topic/consumer 분리, 장애 분류 체계, 대시보드·알림·해결 힌트 흐름 설계와 핵심 구현을 주도했다.

설계 결론

Kafka를 사용했다는 이력보다 중요한 것은 Kafka를 선택한 이유다. 장애 수집은 업무 transaction을 방해하지 않아야 했고, 장애 이벤트는 저장·알림·분석·힌트 제공으로 분기되어야 했다. 이 프로젝트는 운영 데이터가 사후 로그가 아니라 실행 흐름의 일부여야 한다는 기준을 남겼다.